

Introducción a la Ciencia de Datos

Guía de trabajos prácticos N°1

Tabla de contenidos

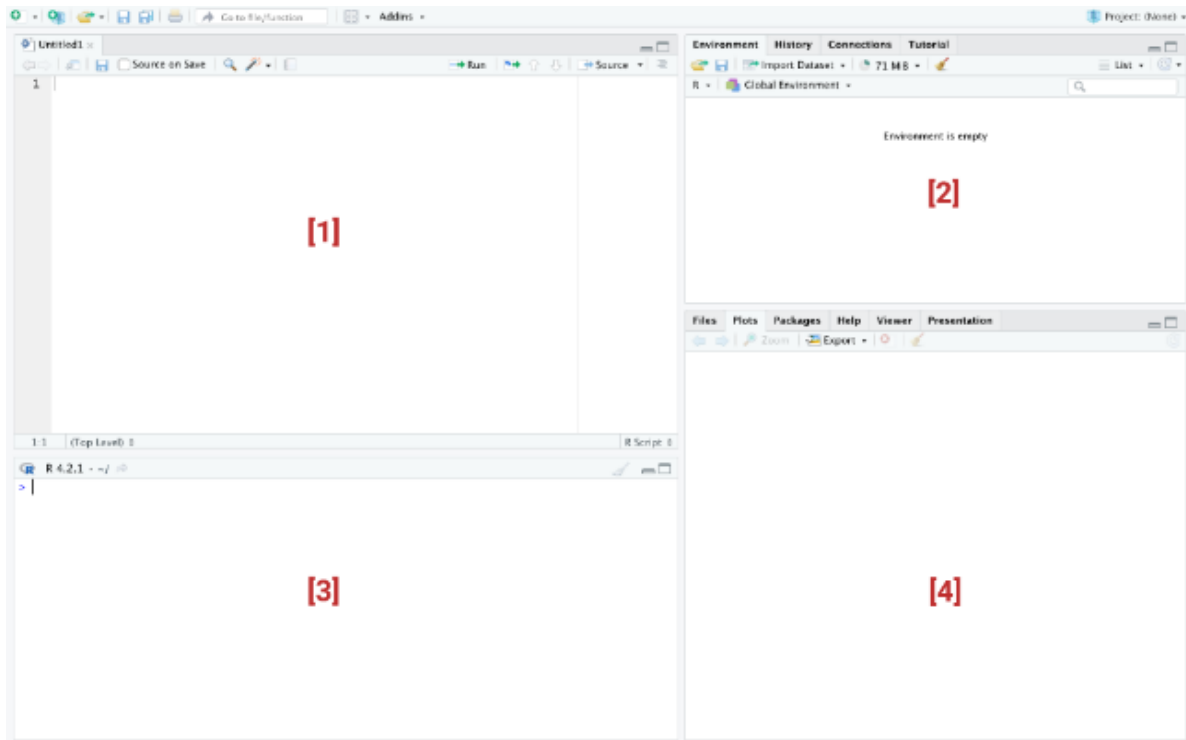
Introducción	1
Conociendo R y RStudio	2
Análisis de precios inmobiliarios con el <i>dataset</i> Properati	3
Levantando los datos	3
Descripción del conjunto de datos	5
Gráfico de dispersión o Scatter plot	7
Tendencias. Hacia el modelado de datos.	8
Análisis del <i>dataset</i> lifestyle	10
Para trabajar en casa	11
Para tener en cuenta antes de ponerse a trabajar	12
Bibliografía obligatoria	12
Bibliografía para profundizar	12

Introducción

Les proponemos comenzar a analizar un conjunto de datos (*dataset*) a partir de una serie de consignas. Para esto vamos a utilizar el lenguaje de programación R, utilizando **RStudio** como entorno de desarrollo integrado (IDE). Si es que no saben de qué se trata, este último término lo verán en detalle en la materia Programación I. Por ahora quedémonos con la idea de que **RStudio** es el programa que vamos a usar para usar **R**. Una vez más, insistimos en que no hace falta saber programación.

Conociendo R y RStudio

1. Abran **RStudio** en la computadora que estén trabajando. Deberían encontrarse con una pantalla como la siguiente:



donde pueden ver 4 ventanas, además de la barra de opciones en la parte superior:

[1] es el editor: se trata del lugar donde escribimos el código para posteriormente ejecutarlo. Al escribir allí no sucederá nada, a no ser que se apriete algún botón para ejecutar los comandos o la combinación de teclas **Ctrl+Enter**. Si cuando abren RStudio esta ventana no les aparece, es porque tienen que crear un nuevo *script* con el botón de arriba a la izquierda en la barra de herramientas (el que tiene una hoja con un +), abrir uno con el típico botón de abrir, o usar el atajo Ctrl+Shift+N.

[2] es el “entorno de trabajo” del programa: en este lugar se muestra el conjunto de datos y los “objetos” (resultados, variables, etc.) que se almacenan al ejecutar diferentes análisis. Cuando aparezcan cosas ahí, pueden cliquearlas para explorarlas.

[3] es la consola. Allí el software ejecuta las operaciones realizadas desde el editor de sintaxis. También se puede escribir y ejecutar código desde allí directamente. (*OJO: no se guarda lo aquí ejecutado en el script que exporten*).

[4] esta zona tiene varias subpestañas: (i) la pestaña *files* es un explorador de archivos como el del sistema; (ii) la pestaña *plots* permite visualizar los gráficos que se generen y visualizar los previos; (iii) la pestaña *packages* permite ver los paquetes descargados y guardados en el disco duro así como gestionar su instalación o actualización; (iv) la pestaña *help* permite acceder a la página oficial del software con recursos para el programa: manuales para el usuario, cursos *online*, información general, descarga de paquetes, información de los paquetes instalados, etc.

La idea es que en esta clase vayan familiarizándose con el uso del lenguaje de análisis de datos R utilizando **RStudio** como interfaz.

Análisis de precios inmobiliarios con el *dataset* Properati

Levantando los datos

2. En RStudio abran un *script* nuevo, en el que vamos a ir desarrollando el trabajo de esta guía. Después pueden salvarlo y subirlo a la nube o bajarlo en un pendrive, para poder conservarlo. Vamos a ir agregando líneas de código y comentando qué hace cada una.
3. Empecemos cargando la biblioteca **tidyverse**, que agrupa varias bibliotecas útiles (fundamentales) para trabajar con datos estructurados, como los que vamos a ver en esta materia.

```
library(tidyverse)
```

Copien esta línea en el script y ejecútenla. Para eso, ubiquen el cursor sobre esa línea, y usen **Ctrl+Enter** (en general, esta combinación sirve para ejecutar el código que hayan seleccionado pintando en el editor, o la línea sobre la que esté parado el cursor, si no hay nada pintado). Al hacer esto, van a ver que aparece el código en la consola, se ejecuta, y tenemos una salida que nos muestra qué paquetes se cargaron. Entre ellos, el que vamos a usar hoy mismo es **ggplot**. A lo largo de la materia iremos usando tareas de distintos módulos de **tidyverse**. También vemos que **R** nos anuncia que dos funciones que ya existían quedaron tapadas por las funciones del mismo nombre que acabamos de cargar: **filter** y **lag**. Pero esto no es muy importante ahora. En general, los comandos para cargar bibliotecas se ubican arriba de todo en los *scripts*.

4. El próximo paso es cargar los datos de Properati. Estos datos están en un archivo de formato csv, como contamos en la parte expositiva de la clase. El archivo pueden encontrarlo linkeado en el campus. Bájenlo y copienlo a la misma carpeta que el

script de R en el que están trabajando y díganle a RStudio que el directorio de trabajo es el mismo que el del script como vieron en la cápsula 0 (Session -> Set Working Directory -> To Source File Location). Copien este código y ejecútenlo (de ahora en más, cada vez que digamos “ejecutar” nos referiremos a usar el atajo Ctrl+Enter en la ventana del editor) .

```
data <- read_csv('properati_SM_SPA.csv')
```

Veamos qué hace esta línea. Acá estamos usando la función `read_csv` del paquete `readr` para leer y cargar el archivo csv. En la jerga, se habla de “levantar” el archivo o de “parsearlo”. El resultado de esa operación, será un objeto que es asignado a la variable `data`. El tipo de objeto no importa mucho, pero si corren el comando `class(data)`, verán que se trata de un objeto de tipo `tbl`, o `tibble`, un objeto específico para tablas de datos.

🔥 ¡Atención!

¡Ojo! Hay otra función, `read.csv` (con punto en lugar de guión bajo) del paquete `utils`, que tiene una función parecida, pero no funciona del todo igual. Usen solo la de `readr`.

i = vs <-

En R es común usar `<-` como el operador de asignación, que es tal vez más claro que el `=` de otros lenguajes. Para escribir más rápido ese símbolo formado por dos caracteres, pueden usar el atajo Alt+- (es decir, la tecla Alt y el menos apretados simultáneamente). De todas formas el `=` también está permitido en R.

i El nombre de las variables

El nombre `data` no tiene nada de especial. Podrían haber usado cualquier nombre. Esto es solo una *referencia* a un objeto en la memoria de la computadora, pero podemos llamar a esa referencia como queramos, dentro de algunas limitaciones impuestas por cada lenguaje. En programación, llamamos a estas referencias variables, pero hay que distinguir eso de las variables que definimos en la clase expositiva. Por otro lado, hay algunas cuestiones generales de buenas prácticas que es importante ir conociendo. Por ejemplo, no es buena idea nombrar a las variables con números, o empezar el nombre de una variable con un número.

Descripción del conjunto de datos

Properati es una empresa inmobiliaria fundada en Buenos Aires en 2012, y que opera en varios países de la región y en EEUU. Hasta hace un tiempo, los datos de los anuncios que aparecían publicados en los sitios web de los diferentes países en los que opera la empresa eran públicos. Esto es un recorte de esos datos, donde solo nos quedamos con las casa y departamentos a la venta en el partido de San Martín, filtramos valores absurdos, y recortamos solo algunos meses de anuncios.

Pregunta

¿Cuál es la unidad de este conjunto de datos?

5. Vamos a empezar a darle un vistazo a la tabla de datos. Para eso, dos funciones muy comunes son

```
view(data)
```

y

```
glimpse(data)
```

Ejecuten cada una de estas líneas y vean el resultado que obtienen.

Comparen los resultados y respondan:

- ¿Ambos métodos permiten ver la totalidad de los datos?
- ¿Qué método permite identificar los nombres de las variables?
- ¿Y los tipos de las variables?

Tip

Otra forma de acceder a la visión de `view` del conjunto de datos es haciendo clic en el nombre de la variable en el “entorno de trabajo”.

Si bien estos comandos nos sirven para poder explorar el *dataset* y entender de qué van, es probable que esto no sea suficiente. Muchas veces deberán recurrir a otras fuentes para entender de qué trata un *dataset*. En el mejor de los casos, el *dataset* tendrá una descripción propia (un README).

Ejecuten el comando

```
help(iris)
```

para obtener información sobre el *dataset* de Iris (unas flores). Esta aparecerá en la ventana [4] de RStudio.

Otras veces, necesitarán herramientas externas. Jamás subestimen una buena búsqueda en **Google**. Esta es una tarea que tendrán que realizar cada vez que se enfrenten a un nuevo *dataset*.

6. Usando **glimpse** y **view** respondan

- ¿Cuántas filas hay? (Ayuda, pueden probar `nrow(data)` también)
 - ¿Cuántas columnas hay?
 - ¿Qué variables son numéricas?
 - ¿Qué otros tipos de variables hay?
 - ¿Cuáles podrían ser categóricas?
7. ¿Qué representa la variable 14 (es una letra 'ele')? ¿cuántos valores toma esta variable? Para responder esto último, usen el comando **unique**, aplicado a la columna. Para indicar una columna del *dataset*, podemos usar una de estas dos formas:

```
data$14  
data["14"]
```

Pero esto no es muy útil, porque hay muchos valores y muchas repeticiones. Ahí entra **unique**. Este comando devuelve una lista con los **elementos únicos** del vector en cuestión. Usen **unique** con la columna 14:

```
unique(data$14)
```

unique pertenece a R base; no hay nada de malo en esto, pero si quieren una versión puramente **tidyverse** (con el paquete **dplyr**), pueden correr:

```
distinct(select(data, 14))
```

Pregunta

¿Qué objeto devuelve cada una de las funciones? Usar la función **class**, como arriba.

Una vez que hayan encontrado que significa 14 usen Google para explorar si su intuición fue cierta. Vuelvan a responder: ¿qué representa la variable 14?

Para ver cuántos anuncios hay para cada valor de la variable 14 pueden usar la función `count`

```
count(data, 14)
```

8. Usen esta misma función para responder: ¿cuántos anuncios de departamentos y cuántos de casas hay en el listado?

Gráfico de dispersión o Scatter plot

Los gráficos de dispersión se utilizan para identificar y mostrar relaciones entre dos **variables numéricas** utilizando puntos para representar cada uno de sus valores. La posición de cada punto en los ejes horizontal y vertical indica los valores de un punto de datos individual. Las relaciones entre las variables pueden describirse de muchas maneras: positivas o negativas, fuertes o débiles, lineales o no lineales.

9. En el próximo punto vamos a pedirles que realicen un gráfico de dispersión (*scatter plot*) del precio de las propiedades (`precio`) vs. la superficie cubierta (`sup_cubierta`). Pero antes, piensen qué tipo de relación esperan encontrar entre estas variables y hagan un gráfico a mano alzada en un papel.
10. Ahora sí, con el comando

```
ggplot(data, aes(x=sup_cubierta, y=precio)) +  
geom_point() +  
xlab("Superficie cubierta [m2]") +  
ylab("Precio [USD]")
```

realicen el gráfico y compárenlo con sus expectativas. ¿Se parece a lo que esperaban? Si la respuesta es no, ¿en qué difiere? ¿por qué puede ser?

Respecto al código que usamos en este caso, lo primero que tuvimos que indicarle al comando `ggplot` es el *data frame* (`data`) y con el comando `aes` (de *aesthetics*, es decir, estéticos) le indicamos qué columna queremos usar como la variable x y cuál otra como la variable y . Con el comando `geom_point()` indicamos el tipo de gráfico que queremos realizar (gráfico “de puntos”) y por último `xlab` e `ylab` nos permiten ponerle nombre a los ejes. Noten que en este caso no sólo estamos dándole un nombre al eje sino que estamos indicando en qué unidades se está midiendo cada una.

11. ¿Cuántas variables se necesitaron para hacer este gráfico?
12. ¿Qué representa cada uno de los puntos del gráfico?
13. ¿Qué información sobre la relación entre las variables brinda este gráfico? ¿Se observa alguna relación clara entre ambas variables? ¿Cómo podrían describir esa relación: creciente, decreciente, o constante? En particular, presten atención a los extremos superior izquierdo e inferior derecho de la figura. ¿Qué notan acerca de la densidad de puntos en estas zonas?

Una modificación común del gráfico de dispersión es la adición de una **tercera variable**. Los valores de la tercera variable pueden codificarse, por ejemplo, modificando la forma o el color de los puntos. En el caso de una tercera variable que indique valores **categoricos** la codificación más común es modificar el color de los puntos. Dar a cada punto un tono distinto de acuerdo a su categoría permite mostrar sencillamente la pertenencia a cada uno de los grupos.

14. Agreguemos al gráfico anterior la información de alguna de las variables categóricas que identificaron en el punto 6. Completen el código de la celda de abajo con el nombre de la variable que quieren mapear al color de los puntos.

```
ggplot(data, aes(x=sup_cubierta, y=precio, color=...)) +  
geom_point() +  
xlab("Superficie cubierta [m2]") +  
ylab("Precio [USD]")
```

La gráfica obtenida debería ser igual a la anterior, excepto porque le agregamos el argumento `color` al comando `aes`, indicando qué columna queremos que tenga en cuenta para colorear los puntos.

15. Repitan los puntos del 9 al 13 pero ahora utilizando otras variables numéricas. Exploren varias opciones para la variable categórica que se codifica al color.

Tendencias. Hacia el modelado de datos.

En la pregunta 12, exploraron la *tendencia* que veían en los gráficos del precio en función de los metros cuadrados de las propiedades. Esa tendencia es el primer **patrón** que descubrimos. Si bien este paso es clave, el resultado tiene que ser en general cuantificado. Esto quiere decir que debemos describir de forma numérica cómo es esa tendencia. Vamos a ver los primeros pasos de ese proceso ahora.

Una opción para acercarnos a la cuantificación de los datos es **ajustar una recta** y reportar los valores de sus parámetros. Esto suena mucho, así que vamos por partes.

En primer lugar, recordemos la ecuación de una recta en el plano:

$$y = mx + b .$$

Esta ecuación nos vincula la variable x con la y usando dos **parámetros**: la ordenada al origen b y la pendiente de la recta m . La ordenada b se interpreta como el valor que toma la variable y cuando $x = 0$. Por otro lado, m nos dice **cómo cambia** el valor de y cuando cambia el valor de x . Cuanto más grande sea la pendiente m mayor será el crecimiento de y para un incremento fijo de x .

La idea entonces es encontrar una recta que describa la tendencia que vemos en los datos de la mejor manera posible. Es decir, encontrar la pendiente m y la ordenada b que mejor describen la relación entre la variable x (en este caso los metros al cuadrado) y la variable y (el precio de la propiedad). Vamos a ver más adelante cómo se encuentra esa recta según algún criterio. El hecho de encontrar los valores que “mejor ajustan” a los datos, se llama “ajustar una recta”.

En R, es muy fácil graficar esta recta (incluso con una cierta banda de error, que discutiremos más adelante), con este comando:

```
ggplot(data, aes(x=sup_cubierta, y=precio)) +  
geom_point() +  
geom_smooth(method='lm') +  
xlab("Superficie cubierta [m2]") +  
ylab("Precio [USD]")
```

Veán que la única diferencia con lo anterior, es la incorporación de la línea `geom_smooth(method='lm')`. Podemos pensar que cada línea que “sumemos” al código, agrega una capa en el gráfico. En este caso, un suavizado (*smooth*) de los datos.

Acá no estamos viendo qué valores de m y b tiene la recta (podríamos), pero vemos con mayor claridad que los precios aumentan a medida que aumenta el metraje de los departamentos (medio obvio, ¿no?). Discutan en equipo / entre equipos qué representa en este caso el valor de la pendiente m .

16. Lo que no resulta tan obvio, es que en realidad, hay dos relaciones marcadamente diferentes para departamentos y casas. Repitan el gráfico anterior, pero usando como variable categórica para colorear a los datos al tipo de propiedad `tipo_propiedad`.

¿Qué tipo de propiedad ve su precio crecer más rápido con el aumento del metraje cuadrado? ¿Pueden dar alguna explicación para esto? Como ven, ya no es tan obvio...

Análisis del *dataset* lifestyle

Se analizaron 44 ciudades de todo el mundo para descubrir dónde es más fácil llevar un estilo de vida sano. Desde los niveles de obesidad hasta los índices de contaminación, cada ciudad ha recibido una puntuación en 10 parámetros de vida saludable. Fuente y detalles del *dataset*: <https://www.lenstore.co.uk/>

En el campus pueden encontrar el archivo **citiesdf.RData**, que es el *dataset* ya en formato data frame listo para ser abierto por RStudio, **no necesitan usar `read_csv(...)`** simplemente aprieten el ícono de **Abrir** en el entorno de trabajo y automáticamente les carga el dataframe *data*. Les proponemos resolver los siguientes puntos utilizando el código que aprendimos en la sección anterior.

17. Mirando el *dataset* responder:

- a. ¿Cuántas observaciones tiene este *dataset*?
- b. ¿Cuántas variables?
- c. ¿Qué representa cada fila? Es decir, ¿cuáles son las **unidades** de este *dataset*? Recuerden que para contestar esta pregunta correctamente es necesario ser lo más específico posible.
- d. ¿Qué variables son categóricas?
- e. ¿Qué variables son numéricas?

18. A partir de los distintos tipos de datos que contiene cada una de las columnas.

- a. Para las variables categóricas: explicar qué representa la variable y cuáles son las posibles categorías dentro de la misma.
- b. Para las variables numéricas: ¿son continuas o discretas? ¿Qué variables tienen formato *float*?

19. ¿Qué variables se podrían usar para hacer un scatter plot? Elijan dos variables cuyo gráfico podría brindarles información interesante. ¿Cómo se imaginan que va a resultar este gráfico?

20. Realizar el gráfico que pensaron en el anterior ¿Qué resultados obtuvieron? ¿Resultó como se habían imaginado? ¿Por qué?

- a. ¿Cómo mejorarían el gráfico?
- b. ¿Es necesario ajustar los límites de los ejes?

- c. ¿Hay algún punto que sobresalga o que les llame la atención?
21. Repetir los dos puntos anteriores para otro par de variables.
 22. De todos los gráficos que fueron haciendo ¿Cuál transmite más información? ¿Identifican algún comportamiento llamativo? ¿Cuál es más claro?.
 23. Elegir uno de los *scatter plots* realizados en el punto anterior al que le agregaremos una “tercera dimensión” coloreando los puntos. Elegir una variable de tal manera que se pueda aprovechar la nueva información que están sumando al gráfico. Pensar cuál y explicar por qué eligieron esa.

Resumen de lo visto en esta Guía:

- Introducción a **RStudio**: editor, entorno de trabajo, consola y panel de archivos/gráficos.
- Cargar una biblioteca con `library()`.
- Cargar un archivo `.csv` con `read_csv`.
- Explorar un *dataset* con `view`, `glimpse`, `nrow` y `class`.
- Seleccionar una columna con `$` y `[...]`; obtener valores únicos con `unique` y `distinct`.
- Contar ocurrencias con `count`.
- Gráficos de dispersión (*scatter plot*) con `ggplot` + `geom_point`.
- Mapear una tercera variable al color de los puntos con `aes(color = ...)`.
- Agregar una recta de ajuste lineal con `geom_smooth(method = 'lm')`.

Para trabajar en casa

Contestá el [Cuestionario de la Clase 1](#) antes de la clase virtual de esta semana.

Instalación de R y RStudio

Pueden seguir los pasos del siguiente link para instalar tanto R como RStudio en sus computadoras.

<https://bookdown.org/jboscomendoza/r-principiantes4/instalacion.html>

Si usan Ubuntu [este](#) script para Ubuntu 22 y [este](#) para Ubuntu 18 ya lo instala con los paquetes necesarios.

Si tienen problemas de instalación, pueden utilizar “Colab”, una herramienta disponible con su drive. Para ello, usen [este](#) enlace y les abrirá un entorno de ejecución de R en la nube.

Para tener en cuenta antes de ponerse a trabajar

Los paquetes que vamos usando en R hay que instalarlos. Por ejemplo, para lo que hicimos en esta guía necesitamos usar el paquete tidyverse que ya habíamos instalado en las compus que usaron en los Laboratorios. Si ese no fuera el caso, antes de eso es necesario instalarlo corriendo en la consola el comando:

```
install.packages("tidyverse")
```

Bibliografía obligatoria

Tipos de datos

<https://bookdown.org/jboscomendoza/r-principiantes4/tipos-de-datos.html>

Cómo trabajar con scatter plots en R

<https://cienciadedatos.github.io/r4ds/03-visualize.html#creando-un-gr%C3%A1fico-con-ggplot>

<https://r-graph-gallery.com/scatterplot.html>

Bibliografía para profundizar

Sobre scatter plots (en inglés):

Overplotting: <https://www.data-to-viz.com/caveat/overplotting.html>

Detractores del Bubble chart: https://www.data-to-viz.com/caveat/area_hard.html

A Complete Guide to Scatter Plots: <https://chartio.com/learn/charts/what-is-a-scatter-plot/#what-is-a-scatter-plot>